A SYNOPSIS ON

WinLogon Forensics: Multi-Auth Artifact Analysis

Submitted in partial fulfilment of the requirement for the award of the degree of

BACHELOR OF TECHNOLOGY IN

COMPUTER SCIENCE & ENGINEERING

Submitted by:

Student Name	**University Roll No.**
Aryan Singh	2023021219
Ashish Kumar	2023021220

Under the Guidance of

Dr. Vimal Kumar

Assistant Professor

Group ID: G30

Department of Computer Science and Engineering

Madan Mohan Malaviya University of Technology

Gorakhpur, Uttar Pradesh February-2026

MMMUT
GORAKHPUR U.P. INDIA
NAAC Grade 'A' University

CANDIDATE'S DECLARATION

We hereby certify that the work which is being presented in the Synopsis entitled **"WinLogon Forensics: Multi-Auth Artifact Analysis"** in partial fulfillment of the requirements for the award of the Degree of Bachelor of Technology in Computer Science and Engineering in the Department of Computer Science and Engineering of the Madan Mohan Malaviya University of Technology, Gorakhpur shall be carried out by the undersigned under the supervision of **Dr. Vimal Kumar, Assistant Professor**, Department of Computer Science and Engineering, Madan Mohan Malaviya University of Technology, Gorakhpur.

Student Name	University Roll No
Aryan Singh	**2023021219**
Ashish Kumar	**2023021220**

The above-mentioned students shall be working under the supervision of the undersigned on

" **WinLogon Forensics: Multi-Auth Artifact Analysis**"

Signature
Supervisor

Signature
Head of the Department

Table of contents

Chapter No.	Description	Page No.
Chapter 1	Introduction and Problem Statement	4-6
Chapter 2	Background Survey	7-8
Chapter 3	Objectives	9
Chapter 4	Hardware and Software Requirements	10-11
Chapter 5	Proposals and Algorithms	12-17
	References	18

CHAPTER 1

Introduction and problem statement

Windows systems record a large amount of information about what users do on the machine. Every time someone tries to log in, starts or ends a session, or requests access, the system creates a record. This includes activity from local accounts, Microsoft accounts, Remote Desktop (RDP) connections, and domain logins handled through Active Directory. For digital forensic analysts, security professionals, and compliance auditors, these records are extremely useful. They help investigators understand what happened on a system, support audit reports, and build accurate timelines during incident response investigations.

However, analysing these records becomes challenging when dealing with large amounts of data. Windows does not store authentication information in a single location; instead, it spreads this data across several sources such as the Security Event Log, application logs, and registry hives. Each of these sources follows its own structure and retention policy. Because of this, investigators often have to manually correlate events from different places, especially when multiple authentication methods are involved. At present, Windows does not provide a single built-in tool that combines and analyses all of this information in one streamlined workflow.

This project addresses that gap. We present a structured extraction and analysis framework targeting Windows authentication artifacts, with a particular emphasis on Event Viewer Security logs. The system automates the collection and parsing of logon-related Event IDs — including 4624 (successful logon), 4625 (failed logon attempt), and associated session events — while distinguishing between logon types such as interactive, network, and remote interactive sessions. Beyond raw extraction, the framework applies temporal correlation to surface behavioural anomalies: repeated failed attempts, off-hours access, and logon patterns inconsistent with established baselines.

1.2 Introduction

Windows authentication logs have long been a cornerstone of security monitoring and forensic work. Each login attempt — local, remote, or through a domain environment — generates a record, and taken together those records tell you exactly who was on a system, when they got in, and how. The data exists. That was never really the issue. The issue is that it's spread across multiple sources, formatted inconsistently, and pulling anything meaningful out of it across different authentication methods takes far more time than most investigations can afford.

The proposed system automates the extraction and correlation of Windows authentication events, pulling from the relevant log sources and organizing the data in a way that makes suspicious activity visible without requiring manual triage. The goal is straightforward — give security teams and forensic investigators a faster path from raw log data to actionable findings.

1.2.1 Windows Login Mechanisms

Windows doesn't rely on a single authentication pathway — it supports several, each leaving its own distinct fingerprint in the Event Log. Local accounts authenticate against the SAM database sitting on the machine itself. Microsoft accounts reach out to cloud identity services. In domain environments, Kerberos handles most of the heavy lifting, with NTLM stepping in as a fallback. RDP adds another layer, establishing remote sessions with their own logon type classifications. What ties all of these together is the Security log — specifically Event ID 4624, which fires on every successful logon, and 4625, which captures every failure. Anyone doing serious forensic work or building a monitoring pipeline needs a working understanding of these mechanisms, because the interpretation of a log entry changes depending on which authentication path produced it.

Each of those log entries carries more than just a timestamp. Embedded in the structured record are the account name, the originating network address, the logon type, the authentication package involved, and associated process metadata. That's genuinely rich data — when you can get to it efficiently. The catch is that the native tooling, Event Viewer, was built for spot-checking, not investigation at scale. Manually scrolling through thousands of entries across multiple machines, trying to reconstruct an access timeline that spans days or weeks, is the kind of task that either gets skipped or produces incomplete results. It's not a workflow problem that better habits fix — it's a tooling gap.

1.2.2 Expected Outcomes

- Automates authentication log extraction from Windows Event Viewer, covering all major logon types and session categories.
- Parses and correlates Security log entries across Event IDs 4624, 4625, 4634, and 4647 to build a complete authentication timeline.
- Flags suspicious access patterns including off-hours logins, repeated authentication failures, and potential privilege escalation activity.
- Produces structured forensic reports and visual summaries suited for security audits and compliance documentation.
- Detects unauthorized access attempts and behaviorally anomalous authentication activity spanning multiple Windows login mechanisms.

1.2.3 Application

- Digital Forensics: Reconstructs user activity timelines from Windows authentication logs to support incident investigation and evidence documentation.
- SIEM Integration: Feeds structured, parsed login data into centralized monitoring platforms, strengthening real-time threat detection pipelines.
- Compliance Auditing: Generates access reports aligned with regulatory frameworks including HIPAA, PCI-DSS, and SOX requirements.
- Insider Threat Detection: Tracks employee access behavior over time, surfacing patterns indicative of unauthorized system access or potential data exfiltration.
- System Administration: Assists administrators in diagnosing authentication failures and refining access control policies through login pattern analysis.

1.3 Problem Statement

Windows logs an enormous volume of authentication activity — every login attempt, session creation, authentication failure, and logoff event, across local accounts, Microsoft accounts, RDP connections, and Active Directory domain authentication alike. On paper, that's exactly the kind of telemetry a forensic investigator or security administrator wants. In practice, accessing it is a different story.

The data lives across multiple Event Viewer channels, stored in binary .evtx format with no standardized tooling to extract and normalize it across different authentication mechanisms in one pass. When an investigator needs to reconstruct an access timeline — particularly one that spans multiple systems or stretches across weeks of activity — they're left working through the native Event Viewer interface manually, sifting through potentially thousands of entries with limited filtering capability. That process is slow under normal circumstances and breaks down entirely under incident response pressure.

The more dangerous attack patterns — slow credential stuffing, staged privilege escalation, cross-machine lateral movement — require correlation that these approaches simply weren't designed to provide. That's the gap this project targets.

CHAPTER 2

Literature Survey

Event log analysis has been a staple of both cybersecurity practice and digital forensics for years. What's shifted recently is the growing role of automation and AI in that process. Manually correlating large volumes of log data, spotting anomalies buried across thousands of entries, and reconstructing accurate forensic timelines are all tasks that don't scale well with human effort alone — and researchers have increasingly turned to machine-driven approaches to close that gap [1][3].

On the detection side, deep learning architectures — particularly LSTM networks and Transformer-based models — have shown strong results when applied to system log data, handling the kind of complex, sequential patterns that simpler methods tend to miss [2]. A parallel thread of research has focused on making those results interpretable. Explainable AI techniques are gaining traction specifically because forensic work has accountability requirements that a black-box output can't satisfy — findings need to be traceable, understandable, and defensible in contexts where legal reliability matters [4].

2.1 Windows Event Log Architecture

The Windows Event Log system has gone through meaningful changes over the years. The classic Event Log API that shipped with Windows XP and Server 2003 was eventually replaced by the modern EVTX format, introduced with Windows Vista and Server 2008 — a shift that brought structured XML-based logging and improved querying capabilities to the platform. Kent and Souppaya's documentation of Windows event logs laid important groundwork for forensic practice, identifying the Security Event IDs most relevant to authentication monitoring and establishing how those entries should be interpreted in an investigative context.

Two Event IDs sit at the center of most authentication analysis work. Event ID 4624 records every successful logon, embedding details about the logon type, the authentication package used, and the originating network address. Event ID 4625 captures the failures — every rejected credential attempt — making it the primary indicator for brute-force activity and credential stuffing campaigns. Alongside these, logon type codes carry their own forensic weight. Type 2 points to an interactive console session, Type 3 to network-based authentication, Type 7 to a screen unlock, and Type 10 to a Remote Desktop connection [8]. Knowing which type produced a given log entry is often what separates a routine access event from a meaningful investigative lead.

Key Contributions:

- Comprehensive documentation of Windows Security Event IDs and their forensic interpretation.
- Classification of logon types and their security implications.
- Understanding of EVTX binary format structure for programmatic log extraction.

2.2 Log Extraction Techniques

Several approaches exist for extracting Windows Event Logs programmatically, each with its own tradeoffs. PowerShell's Get-WinEvent cmdlet is the most accessible native option, supporting XML filtering and XPath queries without any additional dependencies. For environments where platform independence matters, Python libraries like python-evtx and pywin32 offer a viable alternative, enabling .evtx parsing outside of a Windows host entirely.

In forensic work, extraction methodology carries real weight. Timestamped exports and hash verification are routine precisely because collected logs don't stay internal — they can end up as evidence, and their integrity will be questioned [6]. Commercial SIEM platforms approach collection differently, running continuous proprietary collectors that feed into centralized systems. The coverage is good, but the opacity of how those collectors parse and normalize data is a known limitation. When an investigation requires you to explain not just what the data shows but how it was handled, a black-box pipeline is a liability [3].

Advantages:

- PowerShell provides native Windows integration without external dependencies.
- Python libraries enable cross-platform analysis of exported EVTX files.
- XML-based filtering allows precise event selection based on Event IDs and attributes.

Limitations:

- PowerShell scripts require Windows environment and appropriate execution policies.
- Binary EVTX format requires specialized parsing libraries.
- Large log files can cause memory constraints during bulk extraction.

2.3 Comparative Summary of Reviewed Literature

Table 2.3.1: Comparative Analysis of Windows User Access Log Extraction Approaches

Technique	Technology Used	Key Contribution	Limitations
PowerShell-based Extraction	Get-WinEvent cmdlet	Native Windows Integration	Requires Windows Environment
Python EVTX Parsing	Python-evtx library	Cross-platform analysis	Binary Format Complexity
SIEM Integration	Commercial Log Collector	Real-time Monitoring	Expensive licensing costs
TimeLine Analysis	Plaso/Log2Timeline	Forensic Timeline Reconstruction	Complex Setup Requirements

Table 2.3.1 presents a comparative overview of existing approaches for Windows user access log extraction and analysis, highlighting the technologies employed, key contributions, and associated limitations of each technique [1][3][5].

CHAPTER 3

Objective of the Project

3.1 Objective:

Windows logs authentication data across every major login path — local accounts, Microsoft accounts, RDP, Active Directory. That part works fine. The problem shows up the moment you actually need to use that data. Getting it out, making sense of it across four different mechanisms, and producing something an investigator can work with — none of that happens automatically. No native Windows tool covers the full pipeline.

That's the gap this system fills. It handles extraction and parsing across all four login types, runs analysis on the collected data, and produces output that's actually structured for the people using it. Administrators get flagged anomalies instead of raw entries to sort through manually. Investigators get timelines they can reconstruct and stand behind. Compliance teams get formatted reports rather than log dumps. None of this required new data — Windows was already generating everything needed. The system just makes it accessible in a way that holds up under real working conditions.

3.1.1 Sub-objective:

- To implement automated extraction of Windows Security Event Logs using PowerShell Get-WinEvent cmdlet and Python-based EVTX parsing libraries
- To parse and categorize authentication events based on Event IDs (4624, 4625, 4634, 4647, 4778, 4779) and logon types (Interactive, Network, RemoteInteractive)
- To correlate login and logout events for accurate session duration calculation and timeline reconstruction
- To develop pattern recognition algorithms for detecting suspicious authentication behaviors including brute-force attempts, unusual access times, and geographic anomalies
- To create visualization dashboards displaying authentication trends, user access patterns, and security alerts
- To generate comprehensive forensic reports in multiple formats (CSV, JSON, PDF) suitable for security auditing and legal evidence preservation.
- To validate system effectiveness through analysis of real-world Windows Event Log samples and simulated attack scenarios

CHAPTER 4

Software and Hardware Requirements

This chapter describes the hardware and software resources required for the successful development and implementation of the proposed system titled " **WinLogon Forensics: Multi-Auth Artifact Analysis**".

4.1 Software Requirements

4.1.1 Operating System

- Windows 10 / 11 Professional or Enterprise (for Event Log generation and testing)
- Windows Server 2016/2019/2022 (for Active Directory and domain authentication testing)
- Ubuntu 20.04 or higher / Linux (for cross-platform EVTX parsing and analysis)

4.1.2 Programming Language

- Python 3.8 or higher (primary language for log parsing and analysis).
- PowerShell 5.1 or PowerShell Core 7.x (for Windows Event Log Execution)

4.1.2 Development Environment

- Visual Studio Code / PyCharm (primary IDE for Python development)
- Windows PowerShell ISE (for PowerShell script development and testing)
- Jupyter Notebook (for data analysis and visualization prototyping)

4.1.4 Python Libraries and Frameworks

- python-evtx: EVTX binary format parsing library for cross-platform log analysis
- pywin32: Windows API access for direct Event Log querying on Windows systems
- pandas: Data manipulation, aggregation, and time-series analysis of authentication events
- numpy: Numerical computations for statistical anomaly detection
- scikit-learn: Machine learning algorithms for pattern recognition

4.1.5 Data Processing & Analysis

- XML parsing libraries (xml.etree.ElementTree, lxml) for Event Log XML structure parsing
- datetime and dateutil: Timestamp parsing and timezone handling
- re (regular expressions): Pattern matching for log field extraction
- json and csv modules : Data export in standard formats

4.1.6. Visualization Libraries

- matplotlib: Static plots for authentication trends and patterns
- seaborn: Statistical visualization and heatmaps for access patterns
- plotly: Interactive dashboards for temporal analysis and geographic visualization

4.1.7 Database and Storage

- SQLite: Lightweight database for storing parsed authentication events
- PostgreSQL / MySQL (optional): Enterprise-scale deployment with centralized log repository

4.1.8 Report Generation

- ReportLab / PyPDF2: PDF report generation for forensic documentation
- openpyxl: Excel report generation for tabular data export
- Jinja2: Template engine for HTML report generation

4.2 Hardware Requirements

4.2.1 Processor

- Intel Core i5 / AMD Ryzen 5 or higher

4.2.2 Memory (RAM)

- Minimum: 8 GB DDR4
- Recommended: 16 GB or higher

4.2.3 Graphics Processing Unit (GPU)

- NVIDIA GPU with CUDA support (RTX 1050 or higher)

4.2.4 Storage

- Minimum: 100 GB free disk space
- SSD recommended for faster data access

4.2.5 Display

- Standard HD display (1366×768 or higher)

4.2.5 Network

- Ethernet / Wi-Fi connectivity for remote log collection
- Network access to target Windows systems for centralized log extraction

Chapter 5
Possible Methodology

The proposed system employs a multi-stage approach for extracting, parsing, and analyzing Windows authentication logs. The methodology combines automated log extraction using PowerShell and Python, structured data parsing of Windows Event Log XML format, temporal correlation of authentication events, and pattern recognition algorithms to identify security anomalies. Similar automated forensic workflows and AI-assisted log analysis techniques have been discussed in recent digital forensic research [1][3][7]. This comprehensive approach ensures efficient processing of large log volumes while maintaining forensic integrity of the original data [6].

5.1 Approach

The system is built around a multi-stage pipeline that moves authentication log data from raw extraction through to actionable analysis without requiring manual intervention between stages. Extraction is handled through a combination of PowerShell and Python — PowerShell for its native access to the Windows Event Log API, and Python for the parsing flexibility it brings when working with .evtx files programmatically. Once extracted, the raw XML structure of each log entry is parsed into a normalized format, making events from different authentication mechanisms — local, RDP, domain, Microsoft account — comparable against each other in a consistent schema.

From there, the analysis layer takes over. Authentication events are correlated temporally, mapping login and logoff pairs, sequencing failed attempts, and building per-user and per-session timelines across the collected data. Pattern recognition runs over those timelines looking for indicators that warrant attention — repeated failures against the same account, logins outside established access windows, privilege use that doesn't fit normal behavior. The approach draws on automated forensic workflows and AI-assisted anomaly detection methods that have seen growing adoption in recent digital forensics research [1][3][7]. Throughout every stage, the original log data remains untouched — forensic integrity isn't bolted on at the end, it's a constraint the extraction process is built around from the beginning [6].

5.1.1 Log Data Acquisition

The system initiates by identifying target Windows Event Log sources, primarily the Security log channel which contains authentication events. For live systems, PowerShell's Get-WinEvent cmdlet is employed with XML filtering to extract specific Event IDs (4624, 4625, 4634, 4647, 4778, 4779). The extraction script implements administrative privilege checks, handles large log files through batched retrieval, and exports events in XML format for subsequent processing. For forensic analysis of archived logs, the python-evtx library directly parses binary EVTX files, enabling cross-platform analysis without requiring Windows system access. The acquisition phase implements integrity verification through cryptographic hashing (SHA-256) to ensure evidential validity of extracted logs, as recommended in forensic evidence preservation studies [6][8].

Algorithm: Windows Event Log Extraction

Input: Target system, Event ID list, Time range
Output: Extracted event records

1. Initialize system parameters and input configuration.
2. Verify administrative privileges; if privileges are insufficient, terminate execution and log error.
3. Validate input parameters including Event ID list and time range.
4. Identify Windows Security Event Log channel as primary data source.
5. Construct XML-based filter query using provided Event IDs and time constraints.
6. Establish connection with Windows Event Log service.
7. For each Event ID in the input list:
   a. Execute Get-WinEvent query with XML filter.
   b. Retrieve events in batches to handle large log volumes efficiently.
   c. Apply timestamp filtering to retain only events within specified time range.
   d. Parse retrieved records and store temporarily in memory buffer.
8. Merge extracted records from all Event IDs into a unified dataset.
9. Export consolidated events into structured XML format for downstream processing.
10. Perform data validation to remove duplicate or corrupted entries.
11. Compute SHA-256 cryptographic hash of exported log file to preserve forensic integrity.
12. Store extracted logs and integrity hash securely.
13. Generate extraction summary including total records, extraction time, and hash value.
14. Return extracted authentication records along with integrity hash.

End

5.1.2 Event Log Parsing and Normalization

Raw Event Log XML is parsed to extract structured authentication data. Each event record contains nested XML elements specifying timestamp, Event ID, user account, logon type, source IP address, and authentication package. The parser implements field mapping to normalize diverse event structures into a unified data schema. Timestamps are converted to ISO 8601 format with timezone handling, categorical fields are standardized, and missing fields are handled through null-safe processing. The normalized data is structured into records containing fields: timestamp, event_id, event_type, username, domain, logon_type, source_ip, source_hostname, authentication_package, logon_id, and session_id [6][7].

Algorithm: Parse Authentication Events

Input: XML event record
Output: Normalized authentication record

1. Parse XML structure to extract System and EventData nodes
2. Extract timestamp and convert to ISO 8601 format
3. Retrieve Event ID and map to event type:
   - 4624 → Successful Logon
   - 4625 → Failed Logon
   - 4634 → Logoff
   - 4647 → User-Initiated Logoff
4. Extract user account details
5. Retrieve logon type and map to description
6. Extract source IP address and hostname
7. Retrieve authentication package and LogonID
8. Construct normalized record with all extracted fields
9. Return normalized record

END

5.1.3 Authentication Event Correlation

Authentication events are correlated across time to reconstruct complete user sessions. Event ID 4624 entries are paired with corresponding Event ID 4634 or 4647 using the LogonID field as a correlation key. This enables calculation of precise session durations. For Remote Desktop sessions, Event ID 4778 and 4779 provide additional temporal markers. The correlation algorithm implements a state machine tracking active sessions, handles incomplete session records, and resolves concurrent sessions from the same user account through unique LogonID differentiation. Failed authentication attempts (Event ID 4625) are aggregated by username, source IP, and time window to identify potential brute-force patterns.[2]

Algorithm: Correlate Authentication Sessions

Input: List of normalized authentication events
Output: Correlated session records with durations

```
1. Initialize empty session dictionary keyed by LogonID
2. Sort events by timestamp
3. For each event in sorted list:
   If event type is Logon (4624):
     - Create new session entry with logon timestamp
     - Store username, source IP, logon type
     - Add to session dictionary using LogonID as key
   Else if event type is Logoff (4634 or 4647):
     - Lookup corresponding session by LogonID
     - Calculate session duration (logoff - logon time)
     - Mark session as complete
4. Identify incomplete sessions (logon without logoff)
5. Return list of correlated sessions
```

End

5.1.4 Temporal and Behavioral Analysis

The system performs statistical analysis on authentication patterns to establish baseline behavior and detect anomalies. For each user account, temporal features are extracted including typical login hours, average session duration, preferred authentication methods, and common source systems. Anomaly detection algorithms flag deviations such as logins during unusual hours, authentication from new geographic locations, sudden increase in failed login attempts, privilege escalation events, and simultaneous sessions from different IP addresses suggesting credential compromise.[2][5]

5.1.5 Threat Detection and Alert Generation

Detection rules are applied to identify specific threat patterns. Brute-force detection flags accounts with more than a threshold number of failed login attempts within a defined time window. Credential stuffing is detected through rapid authentication attempts across multiple accounts from a single source IP. Lateral movement patterns are identified by analyzing Type 3 (Network) logons accessing administrative shares. Each detected threat generates a prioritized alert with severity classification (Critical, High, Medium, Low) based on confidence score and potential impact. [3][7]

```
Algorithm: Detect Brute-Force Attacks

Input: Failed logon events, Threshold parameters
Output: List of detected brute-force attempts

1. Define parameters:
   - FAILURE_THRESHOLD = 10 attempts
   - TIME_WINDOW = 300 seconds (5 minutes)
2. Group failed logon events by source IP address
3. For each source IP:
   a. Sort failed attempts by timestamp
   b. Apply sliding time window algorithm:
      - Initialize window start at first attempt
      - Count attempts within TIME_WINDOW
      - If count >= FAILURE_THRESHOLD:
        * Flag as brute-force attempt
        * Record affected usernames
        * Calculate attempt rate (failures/minute)

END
```

5.1.6 Forensic Timeline Construction

For investigative purposes, the system constructs detailed authentication timelines. All authentication events are sorted chronologically and presented with contextual information including event type, user account, source system, authentication outcome, and duration (for completed sessions). The timeline visualization enables investigators to reconstruct user activities, identify temporal relationships between events, and correlate authentication patterns with other forensic artifacts. Export functionality generates timeline data in formats compatible with forensic analysis tools. [1][9]

5.1.7 Reporting and Visualization

The system generates comprehensive reports in multiple formats. Executive summaries provide high-level statistics including total authentication events, unique users, successful vs. failed login ratios, most active accounts, and top source IP addresses. Detailed forensic reports include event listings with complete metadata, threat detection findings with evidence chains, and investigative recommendations. Visualizations include time-series graphs of authentication activity, heatmaps showing access patterns, geographic maps of source IP locations, and network diagrams. Reports are exportable in PDF, CSV, JSON, and HTML formats. [3][7]

5.1.8 System Validation and Testing

Validation employs both simulated attack scenarios and real-world Windows Event Log samples. Controlled testing environments generate known authentication patterns including normal user behavior, brute-force attempts, lateral movement, and privilege escalation. The system's detection accuracy is measured through precision, recall, and F1-score. Performance benchmarks assess log processing throughput, memory utilization, and query response times. Validation against public forensic datasets ensures compatibility with diverse Windows versions and authentication configurations. [2][6]

References

[1] **A comprehensive analysis of the role of artificial intelligence and machine learning in modern digital forensics and incident response** – Explores how AI/ML techniques improve pattern recognition, timeline reconstruction, and evidence analysis in digital forensics. *(ScienceDirect, 2023)*

[2] **Advanced system log analyzer for anomaly detection and cyber forensic investigations using LSTM and transformer networks** – Proposes a deep learning framework combining LSTM and Transformer models to automate log anomaly detection and forensic correlation. *(Journal of Cloud Computing, 2025)*

[3] **Digital Forensics and Incident Response (DFIR) Automation: Leveraging AI to Accelerate Breach Investigation** – Discusses the use of AI, NLP, and automated frameworks to speed up evidence collection, log analysis, and incident response workflows. *(JDACM, 2025)*

[4] **Ensuring Transparency in Legal Evidence Analysis** – Focuses on explainable AI (XAI) techniques to make AI-driven forensic decisions interpretable and admissible in legal contexts. *(Forensic Science Journal, 2024)*

[5] **RAGLog: Log Anomaly Detection using Retrieval Augmented Generation** – Uses retrieval-augmented large language models to detect anomalies in system logs and assist automated forensic analysis. *(arXiv, 2023)*

[6] **Digital Forensics Using Artificial Intelligence (AI) and Data Mining** – Examines how AI and data mining uncover patterns and evidence across large digital datasets, enhancing investigative accuracy and speed. *(ResearchGate/Conference Paper, 2024)*

[7] **AI-Enhanced Digital Forensics: Automated Techniques for Efficient Investigation and Evidence Collection** – Reviews automation and AI methods that assist investigators in evidence collection and processing. *(Engineering Journal, 2024)*

[8] **On the inadequacy of open-source application logs for digital forensics** – Evaluates limitations in current logging practices and their impact on forensic evidence quality and investigative readiness. *(ScienceDirect, 2024)*

[9] **Systematic Review: Anti-Forensic Computer Techniques** – Provides a structured review of anti-forensic methods that challenge investigations and outlines countermeasures. *(MDPI Applied Sciences, 2024)*